

8INF138 – Sécurité des réseaux et du Web

Travail pratique #2

Introduction

Les mots de passe sont censés interdire l'accès (à un ordinateur, à un fichier, à un répertoire, à un programme, à un site, à une fonction...), donc plusieurs techniques se sont développées pour les attaquer car, c'est bien connu, il n'y a que l'interdit qui attire, que les portes fermées à ouvrir, que les choses cachées à révéler... De plus, qui ne s'est, un jour, trouvé devant son ordinateur avec un mot de passe oublié ou mal saisi et impossible à retrouver ? Combien d'ordinateurs portables sont protégés par un mot de passe censé rendre leur vol impossible ?

En tout état de cause, les attaques de mots de passe visent :

- Soit à récupérer des mots de passe
- Soit à traverser un contrôle d'accès dont on ne connaît pas le mot de passe.

Les mots de passe sont stockés de 2 manières :

- Mots de passe cryptés
- Chiffre clé (Hash code) des mots de passe

Ils ne devraient jamais être stockés dans un ordinateur, quelle que soit la méthode de stockage. Il est toujours possible de récupérer le/les fichiers les contenant. Combien de temps faut-il à quelqu'un se trouvant dans vos locaux pour brancher une clé USB, faire une recherche de tous les fichiers Windows contenant des mots de passe et les copier sur sa clé puis rentrer tranquillement chez lui ? Et combien de temps faut-il, une fois chez lui pour décrypter le contenu de ces fichiers récupérés ?

Lorsque l'on sait la légèreté des algorithmes de cryptographie des mots de passe de Windows, par exemple, il ne suffit que de quelques secondes pour tous les décrypter. Ils sont pourtant irréversibles, mais on attaquera par dictionnaires ou en force brute. Lorsque l'on sait que la taille moyenne des mots de passe est réduite à 6 ou 7 caractères, parfois 4 ou 5, il suffit de quelques minutes à quelques heures pour les casser. On essaiera toutes les combinaisons possibles de chiffres et de lettres et de signes spéciaux, avec le même algorithme de cryptographie qui a servi à crypter le mot de passe, jusqu'à trouver le chiffre clé et donc le mot de passe utilisé.

L'objectif de ce travail est de permettre aux étudiants :

- d'illustrer la complexité pratique des attaques par recherche par force brute sur le cryptage par mot de passe;
- de tester les compétences des étudiants en matière d'utilisation de primitives de cryptographie symétrique dans les programmes Java;
- de tester les compétences des étudiants dans l'analyse des expériences.

Partie 1 – Introduction : attaques sur les mots de passe

Attaque par table de hachage / hash table attack

- 1.1. Créez un hachage pour n'importe quel mot de passe que vous voulez attaquer. Vous pouvez utiliser <http://www.sha1-online.com/> ou autre source de génération SHA1. Par exemple, le hachage SHA1 pour le mot de passe **MyPassword** est **daa1f31819ed4928fd00e986e6bda6dab6b177dc**
- 1.2. Copiez le hash ci-dessus et accédez à n'importe quel site Web public de table de hachage sha1 (SHA1 decrypter, <https://md5decrypt.net/Sha1> par exemple) et essayez d'obtenir le texte équivalent :



- 1.3. Pouvez-vous faire une attaque de table de hachage pour les mots de passe suivants (hachage SHA1) et en utilisant le même site Web?

N	Mot de passe	Oui je peux / Non je n'y arrive pas
1	P@\$W0rD	
2	Thisismypassword	
3	VeryLongP@\$W0rD	

- 1.4. Quel est le mot de passe le plus court que vous pouvez trouver pour lequel l'attaque de table de hachage ci-dessus a échoué?

Combien de temps pour réaliser une attaque par force brute sur un mot de passe?

Note:

- N'entrez pas vos vrais mots de passe;
- La temps écoulé dépend de la vitesse de traitement;

1.5. Ouvrez les pages <https://www.security.org/how-secure-is-my-password/> et <https://password.kaspersky.com>

1.6. Essayez les mots de passe dans le tableau suivant et vérifiez le temps nécessaire pour les briser par force brute

Mot de passe	Temps sur howsecureismypassword.net	Temps sur le vérificateur de Kaspersky
P@S\$W0rD		
thisismypassword		
VeryLongP@\$W0rD		
%O^t#2Fv0JUjVdRV2RW%		

1.7. Pouvez-vous proposer un mot de passe dont vous vous souviendrez facilement et qui aurait un estimé d'un jour pour le déchiffrer (par l'un ou l'autre des services)?

Partie 2 – Cryptographie

Pour cette partie, nous allons utiliser quelques programmes Java simples mettant en œuvre le chiffrement/déchiffrement DES avec les bibliothèques Java Cryptography Architecture (JCA) / Java Cryptography Extension (JCE).

Encryption DES

Téléchargez, compilez et exécutez les programmes simples suivants (qui sont fournis avec cet énoncé) :

- Encryption DES avec le mode ECB (DES_ECB.java);
- Encryption DES avec le mode CBC et un « inline IV » (DES_CBC.java);
- Encryption DES avec le mode CBC et IV généré par un objet de chiffrement (DES_CBC_getIV.java);

Analysez le code pour voir comment le chiffrement est implémenté.

- 2.1** En utilisant les programmes ci-dessus (et leurs modifications), démontrez l'avantage du mode CBC par rapport au mode ECB

Chiffrement par mot de passe / Password-Based Encryption (PBE)

Notez que dans les exemples de programmes donnés dans la section précédente, les clés utilisées pour le chiffrement ont été générées automatiquement sans aucune entrée de l'utilisateur. Dans de nombreux cas, nous souhaitons implémenter un chiffrement par mot de passe.

Les mécanismes de cryptage basés sur les mots de passe dans JCE sont basés sur des mécanismes de hachage cryptographique. En bref, un mot de passe et un « salt », qui est une donnée aléatoire, sont introduits dans une fonction de mélange (une sorte de fonction de hachage) qui est appliquée de manière itérative le nombre de fois défini par le nombre d'itérations.

Tout cela se produit dans l'objet « Secret Key Factory ». Le résultat est ensuite utilisé pour créer une clé dans l'objet Cipher. Pour déchiffrer, il faut connaître un mot de passe (donnée secrète), le « salt » et le nombre d'itérations (ces deux ne sont généralement pas considérés comme secrets et peuvent être connus de l'attaquant).

- 2.2** Ouvrez, compilez les programmes PBEs.java et Util.java (programme auxiliaire d'utilitaires) et explorez le code PBEs.java afin de découvrir comment on parvient à faire du chiffrement par mot de passe.
- 2.3** Essayez ensuite de changer le nombre d'itérations en une valeur plus petite et voyez comment cela affecte le temps d'exécution
- 2.4** Modifiez le programme afin qu'il puisse déchiffrer le message chiffré.

La mise en œuvre du déchiffrement est très similaire au cas du chiffrement. Vous devez suivre les mêmes étapes préparatoires que pour le chiffrement (y compris l'initialisation du « salt » et du nombre d'itérations, la création du jeu de paramètres PBE, l'initialisation du mot de passe, etc.) La seule différence est que l'initialisation du chiffrement PBE doit être effectuée différemment, comme indiqué

dans le fragment ci-dessous:

```
//Initialiser le chiffrement PBE avec clé et paramètres
pbeCipher.init(Cipher.DECRYPT_MODE, pbeKey, pbeParamSpec);

//déchiffrer le texte chiffré
byte[] plaintext = pbeCipher.doFinal(aCyphertext);
String Plaintext = new String(plaintext);
```

À quelle vitesse le chiffrement DES basé sur mot de passe s'effectue?

Afin d'estimer combien de temps prendrait une attaque de recherche par force brute sur un chiffrement par mot de passe, il faut estimer le temps moyen requis pour effectuer un chiffrement/déchiffrement.

Comment mesurer le temps d'exécution en Java ?

Vous pouvez utiliser soit `System.nanoTime()` pour mesurer le temps écoulé, soit `System.currentTimeMillis()` pour mesurer le temps de l'horloge. Pour une discussion des différences entre les deux avec des exemples de code, consultez [cette page](#).

2.5 Modifiez le fichier `PBEs.java` afin qu'il permette de mesurer le temps d'exécution de :

- La génération de clé et le chiffrement (répété 5 fois)
- La génération de clé et le déchiffrement (répété 5 fois)

Mesurez les temps ci-dessus pour un mot de passe fixe, du texte brut fixe, un « salt » et un nombre d'itérations. Répétez les expériences plusieurs fois pour chaque mesure et enregistrez vos mesures.

2.6 Modifiez votre programme pour essayer différents nombres d'itérations: 1, 5, 10, 100, 1000, 2000... Comment cela affecte-t-il les mesures de temps ci-dessus?

2.7 Essayez différents mots de passe. Les mesures du temps sont-elles affectées?

2.8 Essayez des textes simples de différentes tailles. Les mesures du temps sont-elles affectées?

Estimation du temps requis pour une attaque par recherche par force brute sur un chiffrement par mot de passe

Pour cette section, vous devrez reprendre la liste de mots de passe spécifiée au point 1.3 et votre mot de passe choisi au point 1.4.

2.9 Pour chacun des mots de passe listés au point 1.3 et le mot de passe choisi au point 1.4, estimez le temps nécessaire pour réussir une attaque par recherche par force brute, en supposant qu'un attaquant connaisse:

- le texte en clair prédéfini;
- le texte chiffré produit;
- le « salt »;
- le nombre d'itérations;

Mais pas le mot de passe!

- 2.10** Analysez et étudiez en quoi le temps requis pour l'attaque dépend du nombre d'itérations;
- 2.11** Comparez votre temps estimé avec le temps estimé renvoyé pour les mêmes mots de passe par les services en ligne (section 1.6) et proposez une explication plausible de toute différence observée.

Rapport

- 3.** Rédigez un rapport sur ce qui précède, en fournissant des preuves de vos arguments (par exemple, des extraits de code utilisés pour estimer le temps nécessaire pour un chiffrement, etc.).

Information utile

- Lisez très attentivement la grille de correction qui suit. Celle-ci vous indiquera les sections auxquelles vous devrez répondre ainsi que la pondération de chaque section de votre rapport.
- Aucun IDE ou autre outil de développement spécifique n'est imposé. Tous les exercices peuvent être effectués à l'aide d'éditeurs simples et d'une ligne de commande pour compiler et exécuter des programmes.
- Le rapport du travail devra être clair concis.
- Le rapport en format Word ou PDF doit être remis sur Moodle **avant le 19 novembre 2024 23h59**, ainsi que votre fichier PBEs.java modifié et utilisé pour réaliser votre travail.

Note : dans votre rapport, veuillez documenter l'environnement utilisé et les particularités, s'il y a lieu, de la compilation afin que je parvienne à compiler votre programme.

- Pour vous aider, vous pouvez consulter le guide de référence de JCA, disponible à l'adresse suivante :
<https://docs.oracle.com/javase/8/docs/technotes/guides/security/crypto/CryptoSpec.html>

Grille d'évaluation

Parties et Description	Pondération
Partie 1 – Introduction : attaques sur les mots de passe (25 points)	
Question 1.1	/3
Question 1.2	/3
Question 1.3	/6
Question 1.4	/6
Question 1.5	/1
Question 1.6	/2
Question 1.7	/4
Partie 2 – Cryptographie (75 points)	
Question 2.1	/5
Question 2.2	/7
Question 2.3	/5
Question 2.4	/15
Question 2.5	/10
Question 2.6	/5
Question 2.7	/4
Question 2.8	/4
Question 2.9	/8
Question 2.10	/6
Question 2.11	/6

Annexe

